EXAMEN PRÁCTICO: DESARROLLO DE VIDEOJUEGOS CON UNITY

Proyecto: "Dungeon Survivor 2D - Core Prototype"

Puntaje Total: 100 puntos

- ❌ **PROHIBIDO EL USO DE IA** para los scripts de físicas, movimiento del jugador y sistema de combate (`PlayerController.cs`, `PlayerAttack.cs`, `CameraFollow.cs`).

- 🤖 **PERMITIDO EL USO DE IA ÚNICAMENTE PARA LA UI:** Al igual que en el Taller 6, puedes usar ChatGPT/Claude para apoyarte en la generación de los scripts de interfaz (contador de gemas/monedas en pantalla y menú de pausa).

🎯 OBJETIVO DE LA EVALUACIÓN

Construir una escena jugable desde cero en Unity 2D donde un personaje principal pueda moverse, saltar, ser seguido por la cámara, recolectar gemas con actualización de puntaje en pantalla, atacar y destruir a un enemigo mediante detección física, y pausar la partida mediante interfaz de usuario.

📂 ESTRUCTURA Y PONDERACIÓN

KRAKEDEV
Escuela de Programación

Fase	Área Evaluada	Ponderación
Fase 1	Físicas, Movimiento Horizontal, Salto y Cámara Suave	**30 Pts**
Fase 2	Sistema de Recolección con Trigger y UI (Contador)	**20 Pts**
Fase 3	Combate Melé: Punto de Ataque, Hitbox y Daño al Enemigo	**30 Pts**
Fase 4	Sistema de Pausa In-Game con Congelamiento de Tiempo	**20 Pts**
Total		**100 Pts**

INSTRUCCIONES PASO A PASO

FASE 1 — Físicas, Movimiento y Cámara (30 Puntos)

1. **Configuración de Escena y Físicas:**
    - Crear un objeto `Player` con `SpriteRenderer`, `Rigidbody2D` (marcar **Freeze Rotation Z**) y un `BoxCollider2D`.
    - Crear al menos 2 plataformas (`Platform`) con `BoxCollider2D` y asignarles el Tag: **`Ground`**.
    - Crear un fondo o elementos decorativos asegurando un `Order in Layer` inferior (ej. -10).
2. **Programación del Jugador (`PlayerController.cs`):**
    - Movimiento horizontal continuo con `Input.GetAxis("Horizontal")` multiplicando por una variable pública `moveSpeed`.
    - Salto con la barra espaciadora (`Input.GetButtonDown("Jump")`) aplicando impulso vertical (`jumpForce`).
    - **Control de Suelo:** El salto solo debe ejecutarse si la variable booleana `isGrounded` es `true`. Controlar esto mediante los eventos `OnCollisionEnter2D` y `OnCollisionExit2D` comparando el Tag `"Ground"`.
    - **Volteo Visual (Flip):** Al moverse a la izquierda o derecha, voltear al personaje (modificando `transform.localScale.x` o activando `spriteRenderer.flipX`).
3. **Cámara Suave (`CameraFollow.cs`):**
    - Asignar el script a la `Main Camera`.
    - En el método `LateUpdate()`, hacer que la cámara siga al jugador suavemente usando `Vector3.Lerp()` con un `offset` para mantener al jugador centrado y con `Z = -10`.

FASE 2 — Recolección y Contador UI (20 Puntos)

(Aquí se autoriza el uso de IA para generar o ajustar los scripts).

1. **Prefab de Gema/Moneda:**
    - Crear un objeto `Gema` en la escena con un `CircleCollider2D` o `BoxCollider2D` y marcar la casilla **Is Trigger**.
    - Convertir la moneda en un **Prefab** y colocar mínimo 3 en el escenario.
2. **Interfaz Gráfica (Canvas):**
    - Crear un objeto de texto en la esquina superior de la pantalla (`Text` o `TextMeshPro`) que muestre inicialmente: `Gemas: 0`.
3. **Mecánica de Recolección:**
    - Al entrar el jugador en el Trigger de la gema, el contador en pantalla debe incrementarse en 1 (`Gemas: 1`, `Gemas: 2`, etc.) y el objeto gema debe destruirse de la escena (`Destroy(gameObject)`).

KRAKEDEV Escuela de Programación

FASE 3 — Sistema de Combate Melé (30 Puntos)

1. **El Enemigo (`Enemy`):**
    - Crear un GameObject `Enemigo` con `SpriteRenderer`, `Rigidbody2D` (Freeze Rotation Z) y `BoxCollider2D`.
    - Asignarle la Layer: **`Enemy`**.
    - Programar el script `EnemyHealth.cs` con una vida inicial de 2 o 3 golpes.
    - Implementar el método público `public void TakeDamage(int damage)`, que reste vida y destruya al enemigo (`Destroy(gameObject)`) cuando su salud sea menor o igual a cero.
2. **Punto de Ataque y Hitbox (`PlayerAttack.cs`):**
    - Crear un GameObject hijo vacío dentro del jugador llamado `AttackPoint` ubicado al frente de donde ataca.
    - En `PlayerAttack.cs`:
        - Detectar la pulsación de la tecla **Z** o **Clic Izquierdo**.
        - Utilizar `Physics2D.OverlapCircleAll(attackPoint.position, attackRange, enemyLayer)` para detectar todos los colliders de la capa enemiga alcanzados por el ataque.
        - Mandar a llamar al método `TakeDamage(...)` de cada enemigo detectado.

FASE 4 — Menú de Pausa In-Game (20 Puntos)

(Aquí se autoriza el uso de IA para la UI).

1. **Diseño del Panel de Pausa:**
    - En el Canvas, crear un `PanelPausa` centrado con el texto *"JUEGO EN PAUSA"* y un botón: **"Reanudar"**.
    - Desactivar la casilla del panel en el Inspector para que inicie oculto.
2. **Lógica de Pausa (`PauseMenu.cs`):**
    - Al presionar la tecla **ESC** o **P**:
        - Si el juego está corriendo: Activar el panel y congelar las físicas con `Time.timeScale = 0f`.
        - Si ya estaba pausado: Ocultar el panel y reanudar con `Time.timeScale = 1f`.
    - Conectar el evento `OnClick()` del botón **Reanudar** para cerrar el panel y restaurar `Time.timeScale = 1f`.

KRAKEDEV
Escuela de Programación

CRITERIOS DE ENTREGA

1. **Video Demostrativo de WhatsApp (Máximo 15 segundos):**
    - Grabar la pantalla en modo Play ejecutando:
        1. Movimiento y salto del personaje.
        2. Recolección de al menos una gema y ver cómo cambia el texto en pantalla.
        3. Destrucción del enemigo tras asestarle los golpes con la tecla Z.
        4. Abrir y cerrar el menú de pausa con la tecla ESC.
2. **Repositorio de GitHub:**
    - Subir la carpeta de scripts (`PlayerController.cs`, `CameraFollow.cs`, `PlayerAttack.cs`, `EnemyHealth.cs`, `PauseMenu.cs`, etc.) a tu repositorio.

KRAKEDEV
Escuela de Programación